ChatRealty — Google Ads API Integration

Standard Access Design Document

JPS & Company LLC · Google Ads Manager Account (MCC) 206-304-7113
https://chatrealty.io · June 20, 2026

1. Company & Product Overview

ChatRealty (operated by JPS & Company LLC) is a multi-tenant SaaS marketing platform for licensed U.S. real estate agents. Through our Google Ads manager account (MCC 206-304-7113), ChatRealty creates and manages Search and Video (YouTube) advertising campaigns on each agent's own Google Ads account — on their behalf and with their explicit, revocable consent. ChatRealty is the technology provider; the agent is the advertiser and account owner.

2. Target Audience & Business Model

- Audience: individual licensed real estate agents and small teams in the United States.
- Monetization: subscription tiers plus a prepaid credit system. Agents buy credits (a tier-based markup is applied at purchase only) that fund their ad spend and platform services. ChatRealty does not mark up the ad spend itself — credits convert to ad budget at face value. We never resell Google Ads inventory.

3. System Architecture

- Authentication: each agent connects via OAuth 2.0 (scope https://www.googleapis.com/auth/adwords, access_type=offline). The refresh token is stored encrypted against the agent's user record. Consent is revocable by the agent at any time in Settings → Integrations.
- MCC model: the agent links their existing Google Ads account to ChatRealty's MCC (206-304-7113). API calls set login-customer-id = MCC and customer-id = the agent's account, so all management happens on the agent's own account under their consent.
- Per-request credential isolation: a per-request context (AsyncLocalStorage) threads the acting agent's customerId and refreshToken through every API call, so one agent's request can never touch another's account. The platform developer token is used as the developer-token header; the acting account is always the consenting agent's.
- Stack: Next.js (App Router) and MongoDB; Google Ads API REST (v24).

4. Google Ads API Usage

- Services/methods: CampaignBudgetService (budgets), CampaignService (Search and Video campaigns), AdGroupService, AdGroupAdService (Responsive Search Ads, in-stream Video ads), AssetService (YouTube video assets), and GoogleAdsService.SearchStream for reporting/status.
- Campaign types: Search, Video (YouTube in-stream). Targeting is geo/radius-based for real-estate markets (cities, neighborhoods, listings).
- Volume: modest per-agent operation counts (campaign create, manage, report); well within Standard limits across the agent base.

5. User Workflow

1. Agent connects Google via OAuth and selects their account (auto-discovered with listAccessibleCustomers).
2. Agent links their account to the ChatRealty MCC (a manager link they accept in Google Ads).
3. Agent builds a campaign in the ChatRealty wizard (market, budget, headlines, creative).
4. ChatRealty creates the campaign on the agent's account PAUSED — the agent reviews targeting and bids in Google Ads, then enables it. Nothing spends without the agent's review.
5. ChatRealty reports campaign status and metrics back into the agent's dashboard.

6. Required Minimum Functionality (Capabilities)

- Campaign Creation — Search and Video campaigns, budgets, ad groups, ads.
- Campaign Management — edit/pause/enable, budget changes.
- Reporting — campaign status and performance via GoogleAdsService.
- No Account Creation — agents bring existing accounts. No third-party reselling of inventory.

7. Data Handling, Consent & Policy Compliance

- We access an agent's account only with their OAuth consent and MCC link, both revocable.
- We do not sell, rent, or share agents' Google Ads data; it is used solely to operate their campaigns and show them reporting.
- Campaigns launch PAUSED for human review — no unattended or unauthorized spend.
- We comply with Google Ads policies and the API Terms; agents are responsible for their ad content, and we surface policy/disapproval status to them.

8. Security

- OAuth refresh tokens encrypted at rest; least-privilege scoping (adwords only).
- Per-request account isolation (Section 3) prevents cross-tenant access.
- TLS in transit; access controls on the platform.